

Chain Of Thought

学号：3220100975

姓名：汪珉凯

1. Project Introduction

1.1. 选题

本项目选题为基于 **MindSpore 的思维链**，旨在探索和实现 CoT 推理方法在自然语言处理任务中的应用。通过设计合理的提示模板和推理步骤，提升模型在复杂问题（如数学计算和常识问答）中的推理能力和解释性。

1.2. 工作简介

本实验的主要工作包括以下内容：

- 数据集处理**：对 GSM8K（数学问题）和 CommonsenseQA（常识问答）数据集进行预处理，提取问题文本和参考答案。
- CoT 推理实现**：基于 MindSpore 框架，加载预训练语言模型 **Qwen-2.5-3B**，设计并实现 CoT 推理流程，包括初始提示生成、中间推理步骤生成和最终答案预测。
- 实验评估**：通过对比不同提示模板和触发句的效果，评估模型在算术问题和常识问答任务中的准确率，分析 CoT 方法对模型性能的影响。

1.3. 开发环境及系统运行要求

1.3.1. 开发工具与开发包

- **MindSpore**：用于模型推理和实验实现。
- **Jupyter Notebook**：用于代码开发和调试。
- **AutoModelForCausalLM**：用于加载和管理开源语言模型 Qwen。
- **AutoTokenizer**：用于加载分词器。
- **Pandas**：用于数据集的读取和处理。
- **NumPy**：用于数值计算和结果分析。
- **os**：用于将实验数据记录到txt文件中。

1.3.2. 系统运行要求

- **硬件要求**：
 - 使用 MindSpore
- **软件要求**：
 - Python 3.9 或更高版本。
- **数据集路径**：确保数据集文件（GSM8K 和 CommonsenseQA）位于 **data** 目录下，格式为 parquet。

2. Technical Details

2.1. 理论知识

Chain of Thought (CoT) 是一种通过生成中间推理步骤来引导语言模型解决复杂问题的方法。其核心思想是模拟人类的逐步推理过程，从而提高模型在需要多步逻辑推理任务中的表现。CoT 的关键点包括：

- **中间步骤生成**：模型不仅输出最终答案，还生成解释或推导过程。
- **触发句设计**：通过特定的提示（如 “Let’s think step by step”）引导模型进入逐步推理模式。
- **多步提示构造**：将初始问题、中间推理和答案生成分阶段输入模型，形成完整的推理链。

CoT 的优势在于：

- 提升模型在数学问题、常识推理等任务中的准确性。
- 增强模型输出的可解释性，便于调试和优化。

2.2. 具体算法

CoT 的实现流程分为以下步骤：

1. **初始提示构造**：
 -  为问题文本，[T] 为触发句（如 “Let’s think step by step”）。
2. **中间推理生成**：
 - 将初始提示输入语言模型（LLM），生成中间推理句子 Z。
3. **第二步提示构造**：
 - 结合初始提示和中间推理，构造新提示 “[X0][Z][A]”，其中 [A] 为答案生成模板。算术问题的提示构造为：“Therefore, the answer (arabic numerals) is”；常识问答的提示构造为：“Therefore, among A through E, the answer is”。
4. **答案预测**：
 - 将完整提示输入模型，生成最终答案句子 y。
5. **答案提取与评估**：
 - 从 y 中提取数字或选项字母，与参考答案对比计算准确率。

2.3. 技术细节（重要函数）

2.3.1. 来自于库的重要函数

- **模型加载与推理**：
 - `AutoModelForCausalLM.from_pretrained()`：加载预训练语言模型（如 Qwen-2.5-3B）。
 - `AutoTokenizer.from_pretrained()`：加载对应的分词器，处理输入文本。
 - `model.generate()`：生成模型的输出序列。
- **数据处理**：
 - `pandas.read_parquet()`：读取 GSM8K 或 CommonsenseQA 数据集。
 - `str.split()`：从答案字段中提取参考答案（如分割 “####” 后的内容）。

2.3.2. 自己编写的重要函数（附详细注释）

- **extract_last_number**：提取最终结果（数字）：

```
# 选择最后一个数字作为结果
def extract_last_number(text):
    numbers = re.findall(r'[-?\d+(?:\.\d+)?', text)
    return numbers[-1] if numbers else None
```

- **extract_last_uppercase: 提取最终结果（大写字母）：**

```
# 选择最后一个大写字母作为结果
def extract_last_uppercase(text):
    uppercase_letters = re.findall(r'[A-Z]', text)
    return uppercase_letters[-1] if uppercase_letters else None
```

- **extract_last_assistant_reply: 提取第一次大模型生成的回答：**

```
def extract_last_assistant_reply(text):
    # 通过assistant注释提取出第一次回答的内容
    if 'assistant\n' in text:
        return text.split('assistant\n')[-1].strip()
    if 'assistant' in text:
        return text.split('assistant')[-1].strip()
    return text.strip()
```

- **推理函数: predict_gsm8k_batch (cqa的推理函数只有文件路径不同)：**

```
def predict_gsm8k_batch(questions, gold_labels,
batch_size=16,output_file="gsm8k_training_results.txt"):
    # 如果文件已存在, 先清空内容
    with open(output_file, "w", encoding="utf-8") as f:
        f.write("")

    # questions: 训练集问题
    # gold_labels: 训练集答案
    tokenizer.padding_side = 'left'
    all_predictions = []
    all_outputs = []
    num_questions = len(questions)

    for start_idx in range(0, num_questions, batch_size):
        gc.collect()
        batch_q = questions[start_idx: start_idx + batch_size]
        batch_gold = gold_labels[start_idx: start_idx + batch_size]

        # Step 1: CoT推理
        # messages_step有两部分组成:
        # 1. system类型的消息: 确定模型的角色: helpful assistant, 与训练数
```

据无关。

```

# 2. user类型的消息：用户提问，也就是具体的训练数据。
# CoT的第一步关键在于：在训练数据（q）的基础上，添加一个提示：Let's
think step by step, 这样大模型就会开始第一次推理
messages_step1 = [
    [
        {'role': 'system', 'content': "You are a helpful
assistant."},
        {'role': 'user', 'content': f"{q} Let's think step
by step."}
    ]
    for q in batch_q
]

input_ids_1 = tokenizer.apply_chat_template(
    messages_step1, add_generation_prompt=True,
return_tensors="ms", tokenize=True,
padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_1 = model.generate(input_ids_1, max_new_tokens=512,
temperature=0.7, top_p=0.95) #输出结果
step1_texts = tokenizer.batch_decode(outputs_1,
skip_special_tokens=True) #解码输出为文本
step1_replies = [extract_last_assistant_reply(txt) for txt
in step1_texts] #提取第一个大模型生成的回复

# Step 2: 最终答案
# messages_step2的与messages_step1的区别在于：
# 训练数据在messages_step1的基础上添加了大模型第一次生成的回答，并在
最后添加提示：Therefore, the answer (arabic numerals) is, 这样大模型就会开
始第二次推理
messages_step2 = [
    [
        {'role': 'system', 'content': "You are a helpful
assistant."},
        {'role': 'user', 'content': f"{q} Let's think step
by step. {reply} Therefore, the answer (arabic numerals) is"}
    ]
    for q, reply in zip(batch_q, step1_replies)
]
input_ids_2 = tokenizer.apply_chat_template(
    messages_step2, add_generation_prompt=True,
return_tensors="ms", tokenize=True,
padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_2 = model.generate(input_ids_2, max_new_tokens=256,
temperature=0.7, top_p=0.95) #输出结果
answer_texts = tokenizer.batch_decode(outputs_2,
skip_special_tokens=True) #解码输出为文本
predictions = [extract_last_number(ans) for ans in
answer_texts] #提取最后一个数字作为答案

# 立即打印当前 batch 的结果
for q, gold, output, pred in zip(batch_q, batch_gold,

```

```

answer_texts, predictions):
    print(f"\nQ: {q}\nGold: {gold}\nPred: {pred}\nOutput:
{output}\n")

    # 将当前 batch 的结果追加写入文件
    with open(output_file, "a", encoding="utf-8") as f:
        for q, gold, output, pred in zip(batch_q, batch_gold,
answer_texts, predictions):
            f.write(f"Q: {q}\nGold: {gold}\nPred:
{pred}\nOutput: {output}\n\n")

    all_outputs.extend(answer_texts)
    all_predictions.extend(predictions)

    return all_outputs, all_predictions

```

- **gsm8k的评估函数：evaluate_gsm8k：**

```

def evaluate_gsm8k(max_samples=50, batch_size=16,
data_path='data/gsm8k.parquet'):
    df = pd.read_parquet(data_path)[:max_samples] # 读入前50条数据
    questions = df['question'].tolist() # 仅用列question作为模型输入
    gold_labels = [row['answer'].split("####")[-1].strip() for _, row
in df.iterrows()] # 从 answer 字段中提取"####"之后的内容作为参考答案。
    outputs, predictions = predict_gsm8k_batch(questions, gold_labels,
batch_size=batch_size, output_file="gsm8k_training_results.txt")
    accuracy = sum(pred == gold for pred, gold in zip(predictions,
gold_labels)) / len(gold_labels) #计算推理准确率
    print("GSM8K准确率:", accuracy)
    return outputs, gold_labels, accuracy

```

- **cqa的评估函数：evaluate_cqa**

- 与gsm8k的区别在于数据处理的方式：

- GSM8K:
 - 输入集：仅使用列 question 作为模型输入。
 - Label 集：从 answer 字段中提取"####"之后的内容作为参考答案。
- CQA:
 - 输入集：将 question、choice 与 text 字段拼接作为输入，其中choice和text都要从choices字典中提取得到。
 - Label 集：使用 answerKey 作为正确答案。

```

def evaluate_cqa(max_samples=50, batch_size=16,
data_path='data/cqa.parquet'):
    df = pd.read_parquet(data_path)[:max_samples] # 读入前50条数据
    df['choice'] = df['choices'].apply(lambda x: str(x.get('choice',
'')))
    df['text'] = df['choices'].apply(lambda x: str(x.get('text', '')))

```

```

questions = (df['question'] + " " + df['choice'] + " " +
df['text']).tolist() # 将 question、choices 与 text 字段拼接作为模型输入
gold_labels = df['answerKey'].tolist() # 使用 answerKey 作为正确答案

outputs, predictions = predict_cqa_batch(questions, gold_labels,
batch_size=batch_size ,output_file="cqa_training_results.txt")
accuracy = sum(pred == gold for pred, gold in zip(predictions,
gold_labels)) / len(gold_labels)
print("CQA准确率:", accuracy)
return outputs, gold_labels, accuracy

```

这些函数共同实现了 CoT 的核心流程，包括提示构造、推理生成、答案提取和性能评估。

3. Experiment Results

3.1 算术问题思维链实现：

安装依赖库

导入依赖库

导入模型与分词器

```

5.4 in model_id = "Qwen/Qwen2.5-3B"
tokenizer = AutoTokenizer.from_pretrained(model_id, ms_dtype=ms.float16, mirror='modelscope')
model = AutoModelForCausalLM.from_pretrained(model_id, ms_dtype=ms.float16, mirror='modelscope')
tokenizer.pad_token_id = tokenizer.eos_token_id

100%|██████████| 7.06k/7.06k [00:00<00:00, 7.79MB/s]
100%|██████████| 2.65M/2.65M [00:00<00:00, 36.8MB/s]
100%|██████████| 1.59M/1.59M [00:00<00:00, 40.7MB/s]
100%|██████████| 6.71M/6.71M [00:00<00:00, 42.8MB/s]
683B [00:00, 831kB/s]
100%|██████████| 34.7k/34.7k [00:00<00:00, 18.4MB/s]
Downloading shards: 0%|          | 0/2 [00:00<?, ?it/s]
0%|          | 0.00/3.70G [00:00<?, ?B/s]
0%|          | 1.47M/3.70G [00:00<04:24, 15.0MB/s]
0%|          | 2.98M/3.70G [00:00<04:48, 13.7MB/s]
0%|          | 4.42M/3.70G [00:00<04:36, 14.3MB/s]
0%|          | 5.80M/3.70G [00:00<04:52, 13.5MB/s]
0%|          | 7.10M/3.70G [00:00<05:00, 13.2MB/s]
0%|          | 8.36M/3.70G [00:00<07:02, 9.37MB/s]
0%|          | 9.38M/3.70G [00:00<07:32, 8.75MB/s]
0%|          | 10.3M/3.70G [00:00<08:25, 7.83MB/s]

```

- 此处省略很多行结果代码，最末尾几行结果如下：

```

Downloading shards: 100%|██████████| 2/2 [13:29<00:00, 404.59s/it]
Qwen2ForCausalLM has generative capabilities, as 'prepare_inputs_for_generation' is explicitly overwritten. However, it doesn't directly inherit from 'GenerationMixin'. 'PreTrainedModel' will NOT inherit from 'GenerationMixin', and this model will lose the ability to call 'generate' and other related functions.
- If you are the owner of the model architecture code, please modify your model class such that it inherits from 'GenerationMixin' (after 'PreTrainedModel', otherwise you'll get an exception).
- If you are not the owner of the model architecture class, please contact the model code owner to update it.
Sliding Window Attention is enabled but not implemented for 'eager'; unexpected results may be encountered.
[WARNING] DEVICE(10010,ffff8e2b50b0,python):2025-05-23-20:36:03.830.302 [mindspore/ccsrc/plugin/device/ascend/hal/device/ascend_vmm_adapter.h:149] CheckVmmDriverVersion] Open file /etc/ascend_install.info failed.
[WARNING] DEVICE(10010,ffff8e2b50b0,python):2025-05-23-20:36:03.830.408 [mindspore/ccsrc/plugin/device/ascend/hal/device/ascend_vmm_adapter.h:188] CheckVmmDriverVersion] Driver version is less than 24.0.0, vmm is disabled by default, drvier_version: 23.0.6
Loading checkpoint shards: 100%|██████████| 2/2 [01:43<00:00, 51.85s/it]
100%|██████████| 138/138 [00:00<00:00, 246kB/s]

```

- 导入模型与分词器部分代码运行时间较长，且占用磁盘空间较大。第一次我磁盘空间设置太小，产生了如下错误（后增大磁盘空间得到解决）：

File Save Error for GSM8K.ipynb

Unexpected error while saving file: GSM8K.ipynb [Errno 28] No space left on device: '/home/ma-user/work/GSM8K.ipynb' -> '/home/ma-user/work/.~GSM8K.ipynb'

Dismiss

定义2.3.2提到的所有函数（这里只给出实验截图，不再赘述）

辅助函数

extract_last_number: 提取最终结果（数字）

```
[4] # 选择最后一个数字作为结果
def extract_last_number(text):
    numbers = re.findall(r'-?\d+(?:\.\d+)?', text)
    return numbers[-1] if numbers else None
```

extract_last_assistant_reply: 提取第一次大模型生成的回答

```
def extract_last_assistant_reply(text):
    if 'assistant\n' in text:
        return text.split('assistant\n')[-1].strip()
    if 'assistant' in text:
        return text.split('assistant')[-1].strip()
    return text.strip()
```

推理函数: predict_gsm8k_batch

```
[6] def predict_gsm8k_batch(questions, gold_labels, batch_size=16, output_file="gsm8k_training_results.txt"):
    # 如果文件已存在, 先清空内容
    with open(output_file, "w", encoding="utf-8") as f:
        f.write("")

    # questions: 训练集问题
    # gold_labels: 训练集答案
    tokenizer.padding_side = 'left'
```

评估函数: evaluate_gsm8k

```
def evaluate_gsm8k(max_samples=50, batch_size=16, data_path='data/gsm8k.parquet'):
    df = pd.read_parquet(data_path)[:max_samples] # 读入前50条数据
    questions = df['question'].tolist() # 仅用列question作为模型输入
    gold_labels = [row['answer'].split("###")[-1].strip() for _, row in df.iterrows()] # 从 answer 字段中提取"###"之后的内容作为
    outputs, predictions = predict_gsm8k_batch(questions, gold_labels, batch_size=batch_size, output_file="gsm8k_training_res
    accuracy = sum(pred == gold for pred, gold in zip(predictions, gold_labels)) / len(gold_labels)
    print("GSM8K准确率:", accuracy)
    return outputs, gold_labels, accuracy
```

实验结果分析

- 可以看到结果有以下几个部分组成：
 - Q: 代表着待解决的问题
 - Gold: 实际值
 - Pred: 预测值

- Output：模型推理的整个过程
- Output部分（CoT）具体由以下几个部分组成：
 - system：系统提示（自带）
 - user：模型推理的思维链，也就是在原有问题（James decides to run 3 sprints 3 times a week. He runs 60 meters each sprint. How many total meters does he run a week?）的基础上，添加了大模型第一次生成的回答的提示词（Let's think step by step. ）、大模型第一次生成的回答本身（To calculate the total distance James runs in a week, we need to break down the problem into smaller steps:~~~~~），以及第二次生成的回答的提示词（Therefore, the answer (arabic numerals)）。
 - assistant:最终结果

推理预测成功的案例：

```
Q: James decides to run 3 sprints 3 times a week. He runs 60 meters each sprint. How many total meters does he run a week?
Gold: 540
Pred: 540
Output: system
You are a helpful assistant.
user
James decides to run 3 sprints 3 times a week. He runs 60 meters each sprint. How many total meters does he run a week? Let's think step by step. To calculate the total distance James runs in a week, we need to break down the problem into smaller steps:

1. **Calculate the distance for one sprint:**
   - James runs 60 meters per sprint.

2. **Calculate the total distance for one session (3 sprints):**
   - Since he runs 60 meters per sprint, for 3 sprints he will run:
     \[
     60 \times \text{meters/sprint} \times 3 \text{ sprints} = 180 \text{ meters}
     \]

3. **Calculate the total distance for one week (3 sessions):**
   - If he runs 180 meters per session, for 3 sessions he will run:
     \[
     180 \times \text{meters/session} \times 3 \text{ sessions} = 540 \text{ meters}
     \]

So, the total distance James runs in a week is  $\boxed{540}$  meters. Therefore, the answer (arabic numerals) is
assistant
540
```

- 可以看到，如其输出结果540。

推理预测失败的案例：

```
Q: Carla is downloading a 200 GB file. Normally she can download 2 GB/minute, but 40% of the way through the download, Windows forces a restart to install updates, which takes 20 minutes. Then Carla has to restart the download from the beginning. How long does it take to download the file?
Gold: 160
Pred: 120
Output: system
You are a helpful assistant.
user
Carla is downloading a 200 GB file. Normally she can download 2 GB/minute, but 40% of the way through the download, Windows forces a restart to install updates, which takes 20 minutes. Then Carla has to restart the download from the beginning. How long does it take to download the file? Let's think step by step. Okay, let's break this down into manageable steps:

1. **Calculate the time taken to download the first 80% of the file:**
   - The file size is 200 GB.
   - The first 80% of the file is  $0.8 \times 200 = 160$  GB.
   - Carla's download speed is 2 GB/minute.
   - Time taken to download 160 GB =  $\frac{160 \text{ GB}}{2 \text{ GB/minute}} = 80$  minutes.

2. **Account for the restart and the time taken to restart the download:**
   - A restart takes 20 minutes.
   - After the restart, Carla has to start downloading the remaining 20% of the file.
   - The remaining 20% of the file is  $0.2 \times 200 = 40$  GB.
   - Time taken to download the remaining 40 GB =  $\frac{40 \text{ GB}}{2 \text{ GB/minute}} = 20$  minutes.

3. **Calculate the total time taken:**
   - Time taken to download the first 80% = 80 minutes.
   - Time taken for the restart = 20 minutes.
   - Time taken to download the remaining 20% = 20 minutes.
   - Total time taken =  $80 + 20 + 20 = 120$  minutes.

So, it takes Carla a total of 120 minutes to download the file. Therefore, the answer (arabic numerals) is
assistant
120 minutes.
```

- 可以看到，理论上的答案应该是160，但大模型给出的答案却是120。

最终准确率

GSM8K准确率：0.6

- 可以看到，最终的准确率为0.6。

3.2 CQA思维链实现

代码大部分与GSM8K的实现方式完全一致，只有以下两点不同：

- 大模型第二次生成的提示词不同：

```
# 与gsm8k不同的是: cqa的第二个提示词有不同之处
messages_step2 = [
    [
        {'role': 'system', 'content': "You are a helpful assistant."},
        {'role': 'user', 'content': f"{q} Let's think step by step. {reply} Therefore, among A through E, the answer is"}
    ]
    for q, reply in zip(batch_q, step1_replies)
]
```

- 数据预处理方式

```
def evaluate_cqa(max_samples=50, batch_size=16, data_path='data/cqa.parquet'):
    df = pd.read_parquet(data_path)[:max_samples] # 读入前50条数据
    gold_labels = df['answerKey'].tolist() # 获取答案列表
    questions = df['question'].tolist() # 获取问题列表

    choices = df['choices'].tolist() # 获取 choices 列
    choices_list = []
    for choices_iter in choices:
        labels = choices_iter['label']
        texts = choices_iter['text']
        joined = "".join([f"({label}) {text}" for label, text in zip(labels, texts)])
        choices_list.append(joined)

    input = []
    for question, choice in zip(questions, choices_list):
        input.append(f"Question: {question} Choices: {choice} ")
    questions = input
    outputs, predictions = predict_cqa_batch(questions, gold_labels, batch_size=batch_size, output_file="cqa_training_results.txt")
```

- 除此之外，由于按照原有超参数训练，会出现内存不足（Allocate Memory Failed）的情况，因此调整 batch_size 超参数为4。

实验结果分析

CommonsenseQA准确率：0.56

- 可以看到，最终的准确率约为56%。

3.3 尝试多种模板构建方式与触发句：以算术问题（GSM8K数据集）为例

3.3.1. 改写提示词“Let’s think step by step”为“First, let us analyze the problem carefully.”

- 只需要改变message1和message2的提示词即可：

```
# Step 1: CoT推理
# messages_step有两部分组成:
# 1. system类型的消息: 确定模型的角色: helpful assistant, 与训练数据无关。
# 2. user类型的消息: 用户提问, 也就是具体的训练数据。
# CoT的第一步关键在于: 在训练数据 (q) 的基础上, 添加一个提示: Let's think step by step, 这样大模型就会开始第一次推理
messages_step1 = [
    [
        {'role': 'system', 'content': "You are a helpful assistant."},
        {'role': 'user', 'content': f"{q}First, let us analyze the problem carefully."}
    ]
    for q in batch_q
]

input_ids_1 = tokenizer.apply_chat_template(
    messages_step1, add_generation_prompt=True, return_tensors="ms", tokenize=True,
    padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_1 = model.generate(input_ids_1, max_new_tokens=512, temperature=0.7, top_p=0.95) #输出结果
step1_texts = tokenizer.batch_decode(outputs_1, skip_special_tokens=True) #解码输出为文本
step1_replies = [extract_last_assistant_reply(txt) for txt in step1_texts] #提取第一个大模型生成的回复

# Step 2: 最终答案
# messages_step2的与messages_step1的区别在于:
# 训练数据在messages_step1的基础上添加上了大模型第一次生成的回答, 并在最后添加提示: Therefore, the answer (arabic numerals) is, 这
messages_step2 = [
    [
        {'role': 'system', 'content': "You are a helpful assistant."},
        {'role': 'user', 'content': f"{q}First, let us analyze the problem carefully. {reply} Therefore, the answer ("}
    ]
    for q, reply in zip(batch_q, step1_replies)
]

input_ids_2 = tokenizer.apply_chat_template(
    messages_step2, add_generation_prompt=True, return_tensors="ms", tokenize=True,
    padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_2 = model.generate(input_ids_2, max_new_tokens=256, temperature=0.7, top_p=0.95) #输出结果
answer_texts = tokenizer.batch_decode(outputs_2, skip_special_tokens=True) #解码输出为文本
predictions = [extract_last_number(ans) for ans in answer_texts] #提取最后一个数字作为答案
```

- 按照3.1中的步骤得到如下结果：

```
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdbase  
wpdb  
  
GSM8K准确率： 0.44
```

- 可以看到，改提示词下模型推理的准确率降低为44%%，相较于原先的60%有明显退步。

3.3.2. 改写提示词为“To solve this, we need to consider the following steps:”

- 同理改变对应的提示词：

```
# Step 1: CoT推理
# messages_step1有两部分组成:
# 1. system类型的消息: 确定模型的角色: helpful assistant, 与训练数据无关。
# 2. user类型的消息: 用户提问, 也就是具体的训练数据。
# CoT的第一步关键在于: 在训练数据 (q) 的基础上, 添加一个提示: Let's think step by step, 这样大模型就会开始第一次推理
messages_step1 = [
    {
        'role': 'system', 'content': "You are a helpful assistant.",
        'role': 'user', 'content': f"{q} To solve this, we need to consider the following steps:|'"
    }
]
for q in batch_q

input_ids_1 = tokenizer.apply_chat_template(
    messages_step1, add_generation_prompt=True, return_tensors="ms", tokenize=True,
    padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_1 = model.generate(input_ids_1, max_new_tokens=512, temperature=0.7, top_p=0.95) #输出结果
step1_texts = tokenizer.batch_decode(outputs_1, skip_special_tokens=True) #解码输出为文本
step1_replies = [extract_last_assistant_reply(txt) for txt in step1_texts] #提取第一个大模型生成的回复

# Step 2: 最终答案
# messages_step2的与messages_step1的区别在于:
# 训练数据在messages_step1的基础上添加了大模型第一次生成的回答, 并在最后添加提示: Therefore, the answer (arabic numerals) is, 这
messages_step2 = [
    {
        'role': 'system', 'content': "You are a helpful assistant.",
        'role': 'user', 'content': f"{q} To solve this, we need to consider the following steps: {reply} Therefore,
    }
]
for q, reply in zip(batch_q, step1_replies)

input_ids_2 = tokenizer.apply_chat_template(
    messages_step2, add_generation_prompt=True, return_tensors="ms", tokenize=True,
    padding=True, truncation=True, max_length=1024
) #把消息转化为模型输入
outputs_2 = model.generate(input_ids_2, max_new_tokens=256, temperature=0.7, top_p=0.95) #输出结果
answer_texts = tokenizer.batch_decode(outputs_2, skip_special_tokens=True) #解码输出为文本
predictions = [extract_last_number(ans) for ans in answer_texts] #提取最后一个数字作为答案
```

- 再次按照3.1中的步骤得到如下结果：

```
assistant
Richard lives in an apartment building with 15 floors. Each floor contains 8 units, and 3/4 of the building is occupied. What's the total number of unoccupied units in the building? To solve this, we need to consider the following steps: 1. Calculate the total number of units in the building.
2. Determine the number of occupied units.
3. Calculate the number of unoccupied units by subtracting the number of occupied units from the total number of units.

Let's break it down:

1. **Calculate the total number of units in the building:**
   - There are 15 floors.
   - Each floor has 8 units.
   - Therefore, the total number of units is  $(15 \times 8 = 120)$ .

2. **Determine the number of occupied units:**
   - 3/4 of the building is occupied.
   - So, the number of occupied units is  $(\frac{3}{4} \times 120 = 90)$ .

3. **Calculate the number of unoccupied units:**
   - The number of unoccupied units is the total number of units minus the number of occupied units.
   - Therefore, the number

GSM8K准确率: 0.44
```

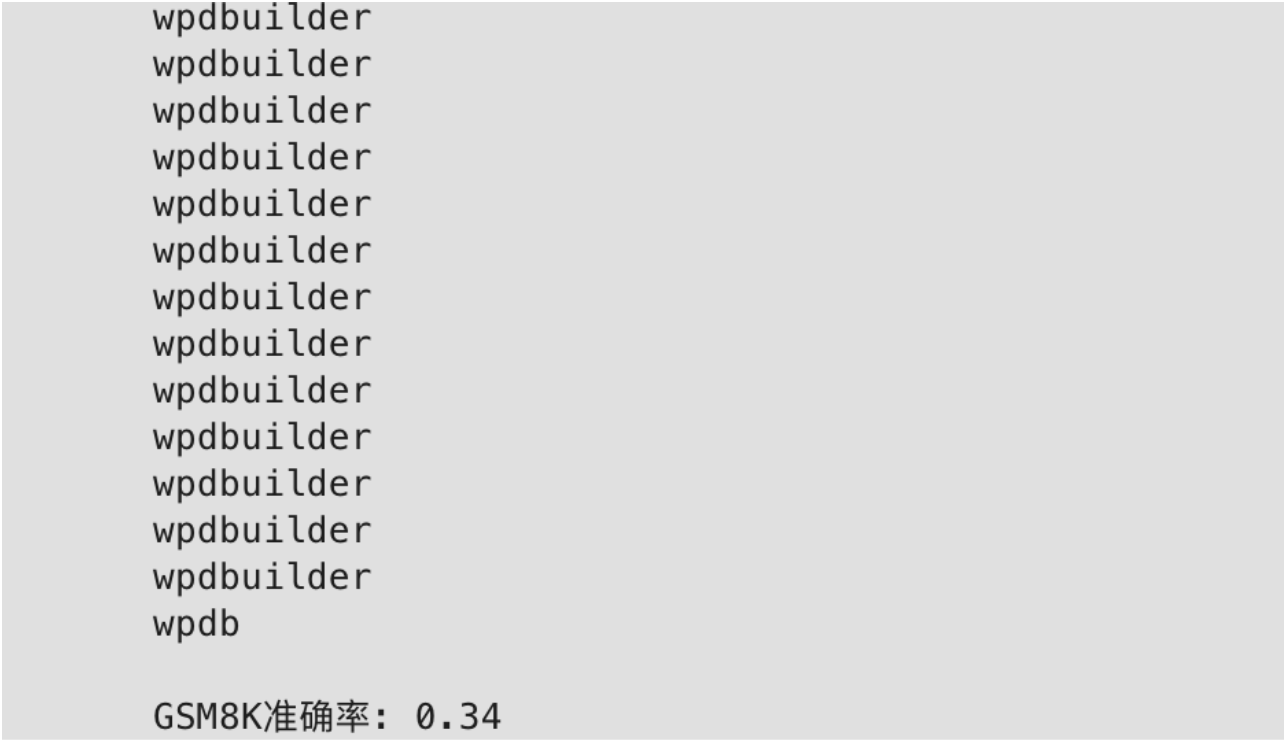
- 可以看到, 改提示词下模型推理的准确率也是44%。

3.3.3. 将提示词改写为如下详细内容：

To solve this mathematical problem correctly:

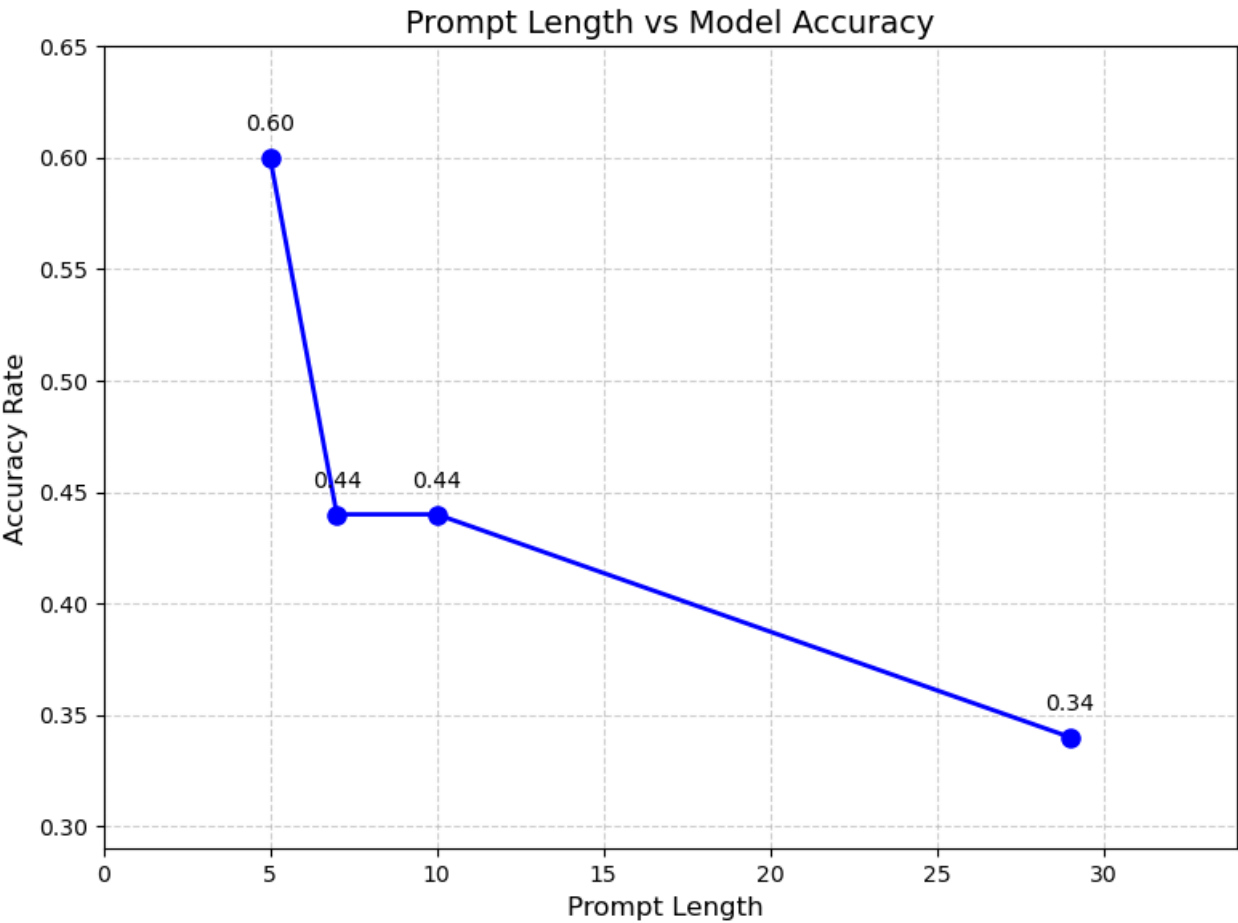
1. Extract given variables and constraints.
2. Recall applicable theorems/formulas.
3. Show intermediate calculations.
4. Check dimensional consistency at each stage.
5. Finalize answer with proper units

- 得到结果如图所示：



3.3.4.对比不同提示词下的准确率（以提示词长度为例）

- 利用matplotlib.pyplot绘制关系图如下：



- 可以看到，模型准确率与提示词长度（复杂度）并无直接联系，与预想的“提示词越复杂效果越好”不一致。

